

CNN Acceleration on the Kria KV260

HLS optimisation of a LeNet-5 CNN

Shuruthy · Kaira · Gilbert

87×

Peak speedup

69.5 μ s

Per-image latency

100%

Accuracy retained

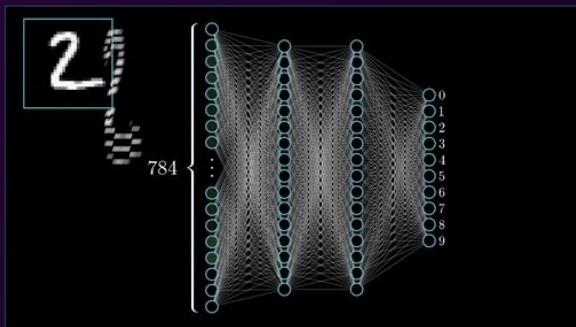
77%

LUT utilisation

Project Overview

Aims

- Implement a LeNet-5 CNN (2 conv + 2 FC layers) for MNIST handwritten digit classification
- Deploy on AMD Kria KV260 (xck26) at 200 MHz using Vitis HLS and the XRT runtime
- Demonstrate a systematic acceleration methodology – profile, identify the bottleneck, target it, iterate
- Support custom hand-drawn digit input for real-time classification on the FPGA



Results

Baseline	6,011 μ s	1 $\times$
Conv accelerated	2,057 μ s	2.9 $\times$
FC unroll $\times 8$	83 μ s	72 $\times$
FC unroll $\times 16$	69.5 μs	87$\times$

Accuracy: 100% at every step both float and ap_fixed<20,8>

Solution

Use of AI tools

- Claude: understanding the HLS pragma semantics, helping to fix II violations and problems with XRT deployment.

01

Profile baseline

02

Identify bottleneck

03

Target with HLS

04

Validate and iterate

Part 1: Initial conv5x5 Optimisation

- Fixed-point (ap_fixed<20,8>) replaces float: enables II=1 pipelining with fewer DSPs
- ARRAY_PARTITION on weight and input buffers: fully parallel BRAM access
- Balanced adder tree: 25 products summed in $\log_2$ stages (5 levels)
- PIPELINE II=1 on output-pixel loop: one result per clock cycle

Part 2: Conv5x5 integration to LeNet

- Fixed-point (ap_fixed<20,8>) replaces float: enables II=1 pipelining with fewer DSPs
- ARRAY_PARTITION on weight and input buffers: fully parallel BRAM access
- Balanced adder tree: 25 products summed in $\log_2$ stages (5 levels)
- PIPELINE II=1 on output-pixel loop: one result per clock cycle

Part 3: conv5x5 integration to LeNet + Other Optimisations (unrolling)

- Fixed-point (ap_fixed<20,8>) replaces float: enables II=1 pipelining with fewer DSPs
- ARRAY_PARTITION on weight and input buffers: fully parallel BRAM access
- PIPELINE II=1 on output-pixel loop: one result per clock cycle
- FC UNROLL factor sweep: $\times 8$, $\times 16$, $\times 32$ tested; $\times 16$ is largest that fits

Solution Architecture: HW/SW

Processing System (ARM A53)

Image loading

Read 784 pixel values from .txt file

MNIST normalisation

$(p/255 - 0.1307) / 0.3081$

XRT DMA transfer

xrt::bo syncs float[784] to PL

Post-processing

softmax $\rightarrow$ probabilities $\rightarrow$ argmax

ARM benchmark

cnn() software reference for speedup

AXI m_axi
float[784] $\rightarrow$

AXI m_axi
 $\leftarrow$ float[10]

Programmable Logic (FPGA)

Pad $1 \times 28 \times 28 \rightarrow 1 \times 32 \times 32$

Conv1 + ReLU $1 \times 32 \times 32 \rightarrow 3 \times 28 \times 28$ *5x5, adder tree, II=1*

MaxPool $3 \times 28 \times 28 \rightarrow 3 \times 14 \times 14$ *2x2 stride*

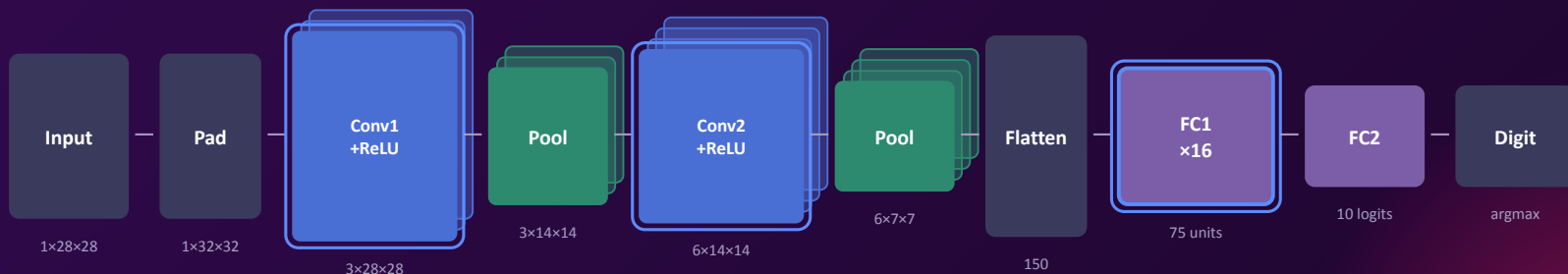
Pad + Conv2 + ReLU $3 \times 14 \times 14 \rightarrow 6 \times 14 \times 14$ *5x5, adder tree, II=1*

MaxPool + Flatten $6 \times 14 \times 14 \rightarrow 294$

FC1 ($\times 16$ unroll) $294 \rightarrow 147$ *16 MACs/cycle*

FC2 $75 \rightarrow 10$ logits

Solution Architecture: LeNet-5 Pipeline



Conv1 & Conv2: how it is accelerated

- 5×5 kernel = 25 multiplications per output pixel
- All 25 computed in parallel – balanced adder tree reduces to $5 \log_2$ stages
- Fixed-point $\text{ap_fixed}\langle 20, 8 \rangle$ replaces float – enables $\text{II}=1$ pipeline
- Weights copied into BRAM at synthesis; no off-chip memory during inference
- Output: one convolved pixel per clock cycle

FC1: the bottleneck

- $294 \text{ inputs} \times 147 \text{ outputs} = 43,218 \text{ MACs per image}$
- Baseline $\text{II}=9 \rightarrow 389\text{k cycles}$: 64% of total V0 time
- Unroll $\times 16$: 16 MACs run in parallel each cycle
- Result: 3700 cycles, 105 $\times$ faster on FC1 alone
- $\times 32$ tried but overflows LUT at 114%, $\times 16$ fits at 77%

Accuracy and precision

- 100% MNIST accuracy – float and fixed-point
- $\text{ap_fixed}\langle 20, 8 \rangle$: 20 bits total, 8-bit integer part
- No accuracy loss vs float at any optimisation step
- Pool + Flatten + FC2 = only 11% of V2 cycles
- FC2 output $\rightarrow$ softmax $\rightarrow$ argmax $\rightarrow$ predicted digit

Performance Evaluation: Method

C-Sim

C-Simulation (g++)

- Compiled with g++ -O2 on host PC. No board needed
- No HLS pragmas, just sequential C++
- Verifies 100 image MNIST accuracy (float)
- RMSE check vs double-precision golden reference

HLS

Vitis HLS C-Synthesis

- Target: xck26 at 200 MHz (5 ns clock)
- csynth.rpt → latency in cycles and μ s
- Reports LUT%, DSP%, FF%, BRAM%
- Checks pipeline II and timing met/failed

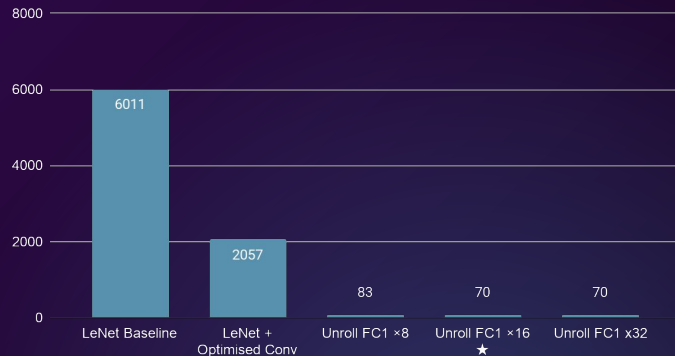
XRT

On-board (KV260)

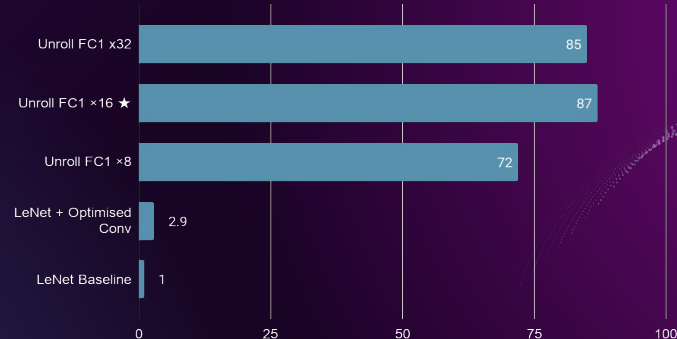
- chrono::high_resolution_clock timing
- 100 MNIST images; 1000 runs for conv5x5
- Custom: ./lenet5_host -x .bin -i digit.txt
- ARM cnn() reference in same binary

Performance Evaluation: Results

Latency per image (μ s)



Speedup



Resource utilisation

Variant	LUT	DSP	BRAM	Fits?
LeNet Baseline	21%	10%	47%	✓
LeNet + Optimised Conv	36%	17%	38%	✓
Unroll FC1 $\times 8$	58%	19%	52%	✓
Unroll FC1 $\times 16$ ★	77%	22%	52%	✓
Unroll FC1 $\times 32$	109%	43%	61%	✗

```
image 97: predicted 7, actual 7
image 98: predicted 6, actual 6
image 99: predicted 9, actual 9
```

FPGA accuracy: 100/100 = 100.00%

--- Performance (avg over 100 images) ---

```
FPGA time : 6705.935 us/image
ARM time : 11547.589 us/image
Speedup : 1.72x
xilinx-kv260-starterkit-20251:~$
```

Baseline LeNet

```
image 97: predicted 7, actual 7
image 98: predicted 6, actual 6
image 99: predicted 9, actual 9
```

FPGA accuracy: 100/100 = 100.00%

--- Performance (avg over 100 images) ---

```
FPGA time : 2356.470 us/image
ARM time : 18301.227 us/image
Speedup : 7.77x
xilinx-kv260-starterkit-20251:~$
```

Optimised Conv5x5 +
baseline LeNet

```
image 97: predicted 7, actual 7
image 98: predicted 6, actual 6
image 99: predicted 9, actual 9
```

FPGA accuracy: 100/100 = 100.00%

--- Performance (avg over 100 images) ---

```
FPGA time : 168.820 us/image
ARM time : 19392.635 us/image
Speedup : 114.87x
xilinx-kv260-starterkit-20251:~$
```

Further optimised LeNet

Discussion

1

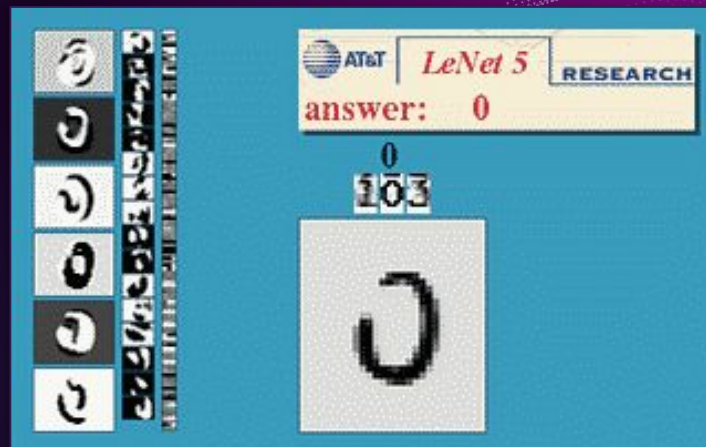
Further optimisations to explore

2

Resource constraints

3

Power usage and utilisation assumptions



Team Progress

Shuruthy Dhushiyandan

- Built the conv5×5 HLS kernel from scratch
- Switched from float to ap_fixed<20,8> for ll=1 pipelining
- Implemented parallel 25-multiplier structure with balanced adder tree
- Ran C-simulation at each step to validate correctness

Kaira Dumasia

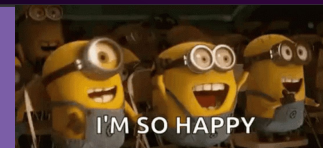
- Simulated bit widths to preserve accuracy through fixed-point conversion
- Produced golden test vectors for C-sim validation
- Extracted and formatted trained weights for HLS use
- Curated and prepared the final demo

Gilbert Tong

- Integrated optimised conv5×5 into the full LeNet architecture
- Optimised FC layers with UNROLL directive, sweeping ×8, ×16, ×32
- Built the Vivado block design and hardware-software integration
- Deployed and tested on the Zynq board end to end



Most of all, we all had so much fun!!!!



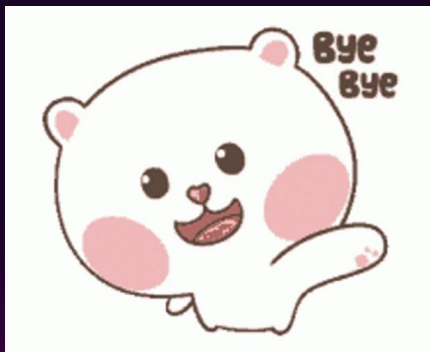
Conclusion

What worked

- `ap_fixed<20,8>` gave $ll=1$ pipelining with zero accuracy loss
- Adder tree: 25-product sum in $5 \log_2$ stages
- FC1 $\times 16$ unroll, single largest gain (53 $\times$ on FC1)
- XRT custom-image mode: hand-drawn $\rightarrow$ real-time prediction
- C-sim flow for rapid iteration without board access

What didn't work

- FC $\times 32$: 109% LUT overflow, cannot place-and-route
- Float-only unroll failed before fixed-point switch
- BRAM inference needed manual array shaping



What we would do differently

- DATAFLOW pragma to overlap layer execution
- Quantisation-aware training $\rightarrow$ `ap_fixed<12,4>`
- `hls::stream<>` to eliminate inter-layer BRAM
- Full 10,000-image on-board benchmark
- Line-buffer for conv memory bandwidth